Введение в обучение с подкреплением

Тема 6: TD методы (продолжение)

Лектор: Кривошеин А.В.

SARSA

SARSA — метод TD-обучения **с единой стратегией,** основанный на оценивании Q-функции.

Базовая схема — обобщённая итерация по ε-мягким стратегиям:

1. **обновляем** оценки Q-функции текущей стратегии π на основе TD метода
   по одношаговому переходу $S_t, A_t, R_{t+1}, S_{t+1}, A_{t+1}$:
   $$Q(S_t, A_t) \leftarrow Q(S_t, A_t) + \alpha[R_{t+1} + \gamma Q(S_{t+1}, A_{t+1}) - Q(S_t, A_t)].$$
2. **улучшаем** стратегию
   $$\pi = \varepsilon\text{–greedy}(Q).$$

SARSA для конечного МППР сходится к оптимальной Q-функции $q_*(s, a)$
при убывающих ε и α.

Q-обучение

Q-обучение (англ. Q-learning) — метод TD-обучения **с разделённой стратегией,**
основанный на оценивании Q-функции, как и SARSA.

Но SARSA работает на основе обобщённой итерации по стратегиям,
а Q-обучение сразу **приближает оптимальные ценности действий** q_*.

Запишем формулу связи V-функции и Q-функции для оптимальной стратегии:

$$q_*(s, a) = \sum_{r \in \mathcal{R}} \sum_{s' \in S} p(s', r \mid s, a)\,(r + \gamma\, v_*(s')).$$

При этом из оптимальности стратегии: $v_*(s) = \max_a q_*(s, a)$.

Следовательно, **уравнение оптимальности Беллмана** для Q-функции имеет вид:

$$q_*(s, a) = \sum_{r \in \mathcal{R}} \sum_{s' \in S} p(s', r \mid s, a)\left(r + \gamma \max_a q_*(s', a)\right) \quad \text{или} \quad q_*(s, a) = \mathbb{E}\left[R_{t+1} + \gamma \max_{a'} q_*(S_{t+1}, a') \,\middle|\, S_t = s, A_t = a\right].$$

Оценку $q_*(s, a)$ можно получить путём усреднения фактически полученных значений
случайной величины $R_{t+1} + \gamma \max_{a'} q_*(S_{t+1}, a')$ для многих траекторий.

Тогда обновление оценки оптимальной Q-функции можно получить в виде:

$$Q(S_t, A_t) \leftarrow Q(S_t, A_t) + \alpha\left[R_{t+1} + \gamma \max_{a'} Q(S_{t+1}, a') - Q(S_t, A_t)\right].$$

Q-обучение

Базовая идея Q-обучения: в ходе взаимодействия агента со средой для каждой полученной четвёрки значений S_t, A_t, R_{t+1}, S_{t+1} обновлять значения Q-функции по формуле

$$Q(S_t, A_t) \leftarrow Q(S_t, A_t) + \alpha\left[R_{t+1} + \gamma \max_{a'} Q(S_{t+1}, a') - Q(S_t, A_t)\right].$$

При этом **не важно по какой стратегии** генерируются эти четвёрки значений S_t, A_t, R_{t+1}, S_{t+1}. Агент может пользоваться некоторой исследовательской стратегией, формула обновления всё равно будет приближать оптимальную Q-функцию.

Сравним эту формулу обновления с обновлением по методу SARSA:

$$Q(S_t, A_t) \leftarrow Q(S_t, A_t) + \alpha[R_{t+1} + \gamma\, Q(S_{t+1}, A_{t+1}) - Q(S_t, A_t)\,].$$

В **SARSA** цель обновления формировалась с помощью текущей стратегии, так как надо выбрать действие A_{t+1}.
В **Q-обучении** цель обновления **не зависит** от используемой стратегии, а формируется с помощью лучшей текущей оценки Q-функции.

Это и позволяет сделать Q-обучения методом обучения **с разделённой стратегией**. То есть генерировать траектории можно по исследовательской стратегии, формируя при этом оценки Q-функции для другой стратегии.

Q-обучение, псевдокод

Приведём псевдокод алгоритма Q-обучения для оценки оптимальной Q-функции q_*. Фиксируем размер шага обучения $\alpha > 0$, а также $\gamma \in (0, 1)$.

---

1. Инициализировать:

значения $Q(s, a) = 0$ для всех состояний $s \in S$ и действий $a \in A$,

задать поведенческую стратегию

2. Q-обучение

Повторять для каждого эпизода:

Выбрать начальное состояние s

Повторять:

Выбрать a, следуя поведенческой стратегии

Наблюдать r, s'

Обновить оценку

$$Q(s, a) := Q(s, a) + \alpha\left[R + \gamma \max_a Q(s', a) - Q(s, a)\right]$$

$s := s'$

Если s заключительное состояние, то выйти из цикла.

Вернуть стратегию π на основе найденой Q-функции.

Условия сходимости алгоритма **Q-обучения**: поведенческая стратегия с потенциально бесконечным числом посещения каждой пары (s, a) + условия Роббинса-Монро на убывание величины шага обучения α.

Q-обучение: особенности

Обсудим выбор поведенческой стратегии.

— Если в ходе взаимодействия со средой агент просто обучается, то поведенческая стратегия может быть любой, лишь бы она обеспечивала выбор каждой пары (s, a) потенциально бесконечное число раз (например, равновероятный выбор действий).

— Если в ходе взаимодействия надо обучаться и одновременно показывать всё более лучшие результаты взаимодействия со средой, то в качестве поведенческой стратегии можно выбрать ε-жадную стратегию относительно Q. Она также обеспечивает выбор каждой пары (s, a) потенциально бесконечное число раз, но чаще будет выбираться действие с наибольшей оценкой.

Преимуществом Q-обучения является то, что обучаться можно на любом наборе сгенерированных траекторий.

Недостатком Q-обучения является тот факт, что при обучении на конечном наборе данных оценки Q-функции будут смещёнными, а именно, завышенными относительно истинных значений.

Q-обучение: особенности

Вид обновления

$$Q(S_t, A_t) \leftarrow Q(S_t, A_t) + \alpha\left[R_{t+1} + \gamma \max_{a'} Q(S_{t+1}, a') - Q(S_t, A_t)\right].$$

Оценки $Q(S_{t+1}, a')$ зашумлены.

Оператор $\max_{a'} Q(S_{t+1}, a')$ выбирает то действие, где оценка $Q(S_{t+1}, a')$ смещена выше истинного значения.

Даже если каждая оценка будет несмещённой в среднем, то максимум по зашумлённым оценкам будем смещён

$$\mathbb{E}\left[\max_{a'} Q(S_{t+1}, a')\right] \geq \max_{a'} \mathbb{E}[Q(S_{t+1}, a')]$$

Q-обучение: особенности

Продемонстрируем эффект завышенных оценок на примере среды с одним состоянием s и некоторым набором действий, возвращающих в состояние s.

Пусть вознаграждения формируются реализацией случайной величины с нормальным распределением $\mathcal{N}(0, 1)$. Тогда истинные оптимальные ценности всех действий $q_*(s, a) = 0$.

Однако, оценки $Q(s, a)$ по методу Q-обучения будут завышены относительно нуля:
проведём симуляцию для 4 действий на 100000 эпизодах, выбор действия равновероятен.

```
Episodes = 100 000; SeedRandom[42]; Acts = 4;
Q = Table[0, Acts];  Num = Table[0, Acts];
For[i = 1, i < Episodes, i++,
 Rew = Table[RandomVariate[NormalDistribution[]], {k, 1, 4}];
 k = RandomChoice[Range[Acts]];
 Num[[k]] = Num[[k]] + 1;
 Q[[k]] = Q[[k]] + (Rew[[k]] + 0.9 Max[Q] - Q[[k]]) / Num[[k]];]
Q
```

```
{0.948804, 0.935933, 0.933287, 0.944938}
```

Конечно же чем больше данных доступно, тем меньше будет это смещение. В пределе смещение исчезнет.

```
For[i = 1, i < Episodes, i++,
 Rew = Table[RandomVariate[NormalDistribution[]], {k, 1, 4}];
 k = RandomChoice[Range[Acts]];
 Num[[k]] = Num[[k]] + 1;
 Q[[k]] = Q[[k]] + (Rew[[k]] + 0.9 Max[Q] - Q[[k]]) / Num[[k]];]
Q
```

```
{0.887875, 0.881954, 0.872695, 0.882199}
```

Q-обучение: особенности

Если же истинные значения ценностей различны, то завышение оценок может помешать сразу выявить самое ценное действие.

Проведём симуляцию для 2 действий a_1, a_2 на 10000 эпизодах, выбирая действия по ε-жадной стратегии.

За действие a_1 вознаграждение всегда $R = 0$, $q_*(s, a) = 0$.

За действие a_2 вознаграждение $R \sim \mathcal{N}(-0.2, 1)$, $q_*(s, a) = -0.2$.

```
Episodes = 10000; SeedRandom[14]; Acts = 2;
Q = Table[0, Acts]; Num = Table[0, Acts];
For[i = 1, i < Episodes, i++,
 Rew = {0, RandomVariate[NormalDistribution[]] - 0.2};
 If[RandomReal[] > 0.01 ,
      k = First@Flatten[Position[Q, Max[Q]]],
      k = RandomChoice[Range[Acts]]];
 Num[[k]] = Num[[k]] + 1;
 Q[[k]] = Q[[k]] + (Rew[[k]] + 0.9 Max[Q] - Q[[k]])/Num[[k]];]
Print["Оценки ценности действий:
Q(s,a₁)=", Q[[1]], "
Q(s,a₂)=", Q[[2]]]
```

```
Оценки ценности действий:
Q(s,a₁)=0.0497841
Q(s,a₂)=0.353094
```

С увеличением объёма данных истинно ценное действие будет обнаружено.

```
For[i = 1, i < 350000, i++,
 Rew = {0, RandomVariate[NormalDistribution[]] - 0.2};
 If[RandomReal[] > 0.01 ,
     k = First@Flatten[Position[Q, Max[Q]]],
     k = RandomChoice[Range[Acts]]];
 Num[[k]] = Num[[k]] + 1;
 Q[[k]] = Q[[k]] + (Rew[[k]] + 0.9 Max[Q] - Q[[k]])/Num[[k]];]
Print["Оценки ценности действий:
Q(s,a1)=", Q[[1]], "
Q(s,a2)=", Q[[2]]]
```

```
Оценки ценности действий:
Q(s,a1)=0.0379957
Q(s,a2)=0.0362189
```

Двойное Q-обучение

Рассмотрим простой способ снижения эффекта завышения оценок при Q-обучении:

двойное Q-обучение (англ. Double Q-learning).

Базовая идея: будем использовать две Q-функции Q_1 и Q_2.

После каждого временного шага при обновлении оценок с вероятностью $\frac{1}{2}$ будем выбирать из двух Q-функций Q_1 или Q_2, которую будем обновлять.

Рассмотрим обновление для Q_1 и будем формировать цель обновления с помощью Q_2

$$Q_1(s, a) := Q_1(s, a) + \alpha\left(R + \gamma\, Q_2\left(s', \operatorname{argmax}_a Q_1(s', a)\right) - Q_1(s, a)\right).$$

То есть вторая Q-функция используется, чтобы дать оценку тому действию, которое максимизирует Q_1.

Аналогичным образом обновляем Q_2.

Само действие a в текущем состоянии s выбирается ε-жадно по значениями суммы Q-функций Q_1 и Q_2, то есть

$a = \varepsilon\text{–greedy}(Q_1(s, \cdot) + Q_2(s, \cdot))$.

Итоговую оценку Q-функции можно получить в виде усреднённого значения Q-функций: $\frac{Q_1+Q_2}{2}$.

Двойное Q-обучение

1. Инициализация

$Q_1(s, a)$, $Q_2(s, a) = 0$ для всех s, a.

2. Повторять для каждого эпизода

Выбор начального состояния s

Повтор для каждого шага эпизода

Выбрать a ε-жадно по $Q_1(s, \cdot) + Q_2(s, \cdot)$

Наблюдать r, s'

Если $x \sim N(0, 1) > 0$, то

$$Q_1(s, a) := Q_1(s, a) + \alpha\left(R + \gamma Q_2\left(s', \operatorname*{argmax}_a Q_1(s', a)\right) - Q_1(s, a)\right)$$

Иначе

$$Q_2(s, a) := Q_2(s, a) + \alpha\left(R + \gamma Q_1\left(s', \operatorname*{argmax}_a Q_2(s', a)\right) - Q_2(s, a)\right)$$

$s := s'$

пока s не станет заключительным.

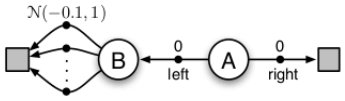
Возврат оптимальной стратегии π на основе Q функции.

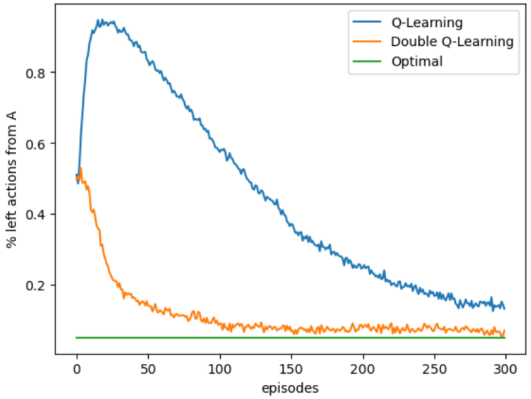
Эффект завышения оценок будет снижен, но удваиваются затраты на память и снижается эффективность использования данных, так как надо обновлять две оценки.

Двойное Q-обучение

Сравним Q-обучение и Двойное Q-обучение на простом примере:

- У состояния A два действия:
  - вправо — награда 0 и завершение
  - налево — награда 0 и новое состояние B
- У состояния B 10 действий:
  - за каждое действие даётся награда из нормального распределения $\mathcal{N}(-0.1, 1)$.





Expected SARSA

Рассмотрим гибрид методов Q-обучения и SARSA называемый **Expected SARSA**. Пусть траектория взаимодействия агента со средой формируется по некоторой поведенческой стратегии π_b. Рассмотрим ещё одну стратегию π (она может и совпадать с π_b) и правило обновления:

$$Q(S_t, A_t) \leftarrow Q(S_t, A_t) + \alpha\left[R_{t+1} + \gamma \sum_{a \in \mathcal{A}} \pi(a \mid S_{t+1})\, Q(S_{t+1}, a) - Q(S_t, A_t)\right]$$

В этом правиле цель обновления формируется по-новому.

Вместо максимума $\max_a Q(S_{t+1}, a)$ как в **Q-обучении** или

вместо текущей оценки $Q(S_{t+1}, A_{t+1})$, где A_{t+1} формируется по стратегии π_b, как в **SARSA**,

для формирования цели обновления используется усреднение оценок $Q(S_{t+1}, a)$ по возможным действиям с вероятностями выбора этих действий в рамках стратегии π.

Что же оценивает Q-функция с таким правилом обновления? Напомним уравнение Беллмана для функции ценности действий стратегии π и запишем его с помощью математического ожидания:

$$q_\pi(s, a) = \sum_{r \in \mathcal{R}} \sum_{s' \in S} p(s', r \mid s, a)\left(r + \gamma \sum_{a' \in \mathcal{A}} \pi(a' \mid s')\, q_\pi(s', a')\right) = \mathbb{E}\left[R_{t+1} + \gamma \sum_{a' \in \mathcal{A}} \pi(a' \mid S_{t+1})\, q_\pi(S_{t+1}, a') \,\middle|\, S_t = s, A_t = a\right].$$

Expected SARSA

Таким образом, Expected SARSA с указанным выше правилом обновления оценивает функцию ценности действий $q_\pi(s, a)$. При этом не важно откуда получаются отсчёты для правила обновления, усреднение проводится по всем возможным будущим вознаграждениям R_{t+1} и состояниям S_{t+1}.

То есть Expected SARSA можно запускать как метод с **единой стратегией** и как метод с **разделённой стратегией**, где поведенческая стратегия для генерации траектории более исследовательская, а целевая стратегия — это ε-жадная или жадная стратегия.

Известно, что дисперсия оценок по методу Expected SARSA меньше, чем дисперсия оценок по методу SARSA из-за того, что в SARSA цель обновления формируется по текущей оценке $Q(S_{t+1}, A_{t+1})$ и для разных действий может быть большой разброс оценок, а в Expected SARSA высчитывается ожидание по целевой стратегии.

Однако, Expected SARSA, чуть более вычислительно затратно, чем SARSA.

Expected SARSA, псевдокод

Приведём псевдокод для метода Expected SARSA с единой ε-жадной стратегией.

---

1. Инициализировать:

значения $Q(s, a) = 0$ для всех состояний $s \in \mathcal{S}$ и действий $a \in \mathcal{A}$,

стратегию π определить ε-жадно относительно Q-функции

2. Expected SARSA

Повторять для каждого эпизода:

Выбрать начальное состояние s

Повторять:

Выбрать a, следуя ε-жадной стратегии относительно Q-функции

Наблюдать r, s'

Обновить оценку

$$Q(s, a) := Q(s, a) + \alpha\left[R + \gamma \sum_{a \in \mathcal{A}} \pi(a \mid s')\, Q(s', a) - Q(s, a)\right]$$

$s := s'$

Если s заключительное состояние, то выйти из цикла.

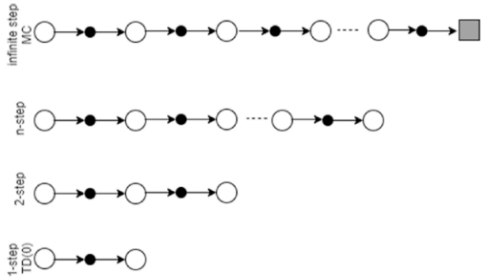
Уменьшить ε.

Вернуть стратегию π на основе найденой Q-функции.

Условия сходимости аналогичны условиям сходимости для метода SARSA (GLIE + условия Роббинса-Монро).

TD: n-шаговые методы

TD метод является 1-шаговым методом, а метод МК является ∞-шаговым методом.



Можно естественным образом привести n-шаговый алгоритм SARSA и Expected SARSA с единой стратегией, $n \geq 1$.
Цель обновления для n-шагового SARSA на k-ой итерации будет иметь вид:

$G_{t:t+n} = R_{t+1} + \gamma R_{t+2} + \ldots + \gamma^{n-1} R_{t+n} + \gamma^n Q_k(S_{t+n}, A_{t+n})$, если $0 \leq t < T - n$,

$G_{t:t+n} = G_t$, если $t \geq T - n$.

Чтобы обновить оценку для текущей пары (S_t, A_t) надо подождать n шагов, сформировать цель обновления и

$Q(S_t, A_t) \leftarrow Q(S_t, A_t) + \alpha (G_{t:t+n} - Q(S_t, A_t))$. При $n = 1$ это обновление соответствует обычному SARSA.

TD: n-шаговые методы

Приведём псевдокод n-шагового SARSA с единой ε-жадной стратегией, $\alpha > 0$, $\varepsilon > 0$, $0 < \gamma \leq 1$, а также $n \geq 1$.
В псевдокоде за S, A, R будем обозначать **очереди длины n** для хранения состояний, действия и вознаграждений.
Добавление элементов всегда происходит у конец очереди, причём первый элемент вылетает, если длина превышает n.

---

1. Инициализировать: значения $Q(s, a) = 0$ для всех состояний s и действий a

2. n-шаговый SARSA

Повторять для каждого эпизода:

В начальном состоянии s выбрать действие a ε-жадно относительно Q-функции

Повторять, пока not done:

Сделать действие a, наблюдать r, s', done

Выбрать действие a' в состоянии s' ε-жадно относительно Q-функции

Добавить в конец очереди: s в S, a в A, r в R

Если длина(S) == n:

$$G \leftarrow \sum_{i=0}^{n-1} \gamma^i R[i]$$

Если not done, то $G \leftarrow G + \gamma^n Q(s', a')$

Обновить оценку $Q(S[0], A[0]) \leftarrow Q(S[0], A[0]) + \alpha[G - Q(S[0], A[0])]$

$s, a = s', a'$

Для $j = 1, \ldots, n$:

$$G \leftarrow \sum_{i=j}^{n-1} \gamma^{i-j} R[i]$$

Обновить оценку $Q(S[j], A[j]) \leftarrow Q(S[j], A[j]) + \alpha[G - Q(S[j], A[j])]$

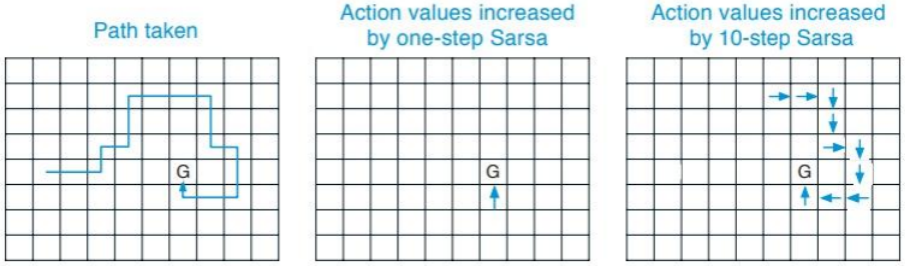
Уменьшить ε

TD: n-шаговые методы

Условия для сходимости n-шагового SARSA аналогичны SARSA:

GLIE + условия Роббинса-Монро на шаг обучения.

Ниже демонстрируется сравнение 1-шагового и 10-шагового SARSA



Стрелки на рисунках указывают, какие ценности действий были изменены в случае одного и второго методов.

Одношаговый метод повышает ценность лишь одного последнего действия.

n-Шаговый SARSA обновляет ценности последних n действий, что повышает эффективность обучения в рамках одного эпизода.

TD(λ)

Метод TD (λ) прямого обзора (англ. forward view), позволяет объединить в себе ∞-шаговые, n-шаговые и 1-шаговые обновления.

Для текущего момента времени t найдём доход за n шагов вперёд (с бутстреппингом):

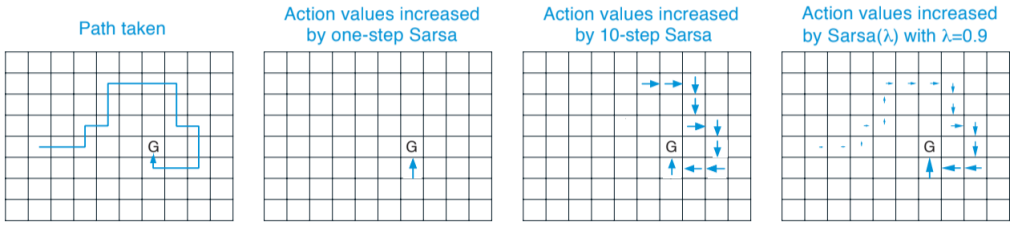
$$G_{t:t+n} = R_{t+1} + \gamma R_{t+2} + \ldots + \gamma^{n-1} R_{t+n} + \gamma^n V(S_{t+n}).$$

Введём новую цель обновления G_t^λ, которое будет взвешенным усреднением всех n-шаговых доходов:

$$G_t^\lambda = (1-\lambda) \sum_{n=1}^{\infty} \lambda^{n-1} G_{t:t+n} = (1-\lambda) G_{t:t+1} + (1-\lambda)\lambda G_{t:t+2} + \ldots$$

$$G_{t:T}^\lambda = (1-\lambda) \sum_{n=1}^{T-t-1} \lambda^{n-1} G_{t:t+n} + \lambda^{T-t-1} G_{t:T}, \quad \text{где } T \text{ момент завершения эпизода.}$$

При $\lambda = 0$ мы получим TD(0) — это одношаговый TD. При $\lambda = 1$ получим метод МК.



TD(λ)

Для вычисления

$$G_{t:T}^{\lambda} = (1-\lambda) \sum_{n=1}^{T-t-1} \lambda^{n-1} G_{t:t+n} + \lambda^{T-t-1} G_{t:T}$$

агенту придётся дожидаться конца эпизода, чтобы сделать обновление, так как нужна информация от каждого шага.

Существует трюк, позволяющий проводить обновления он-лайн.

Основная идея: из текущего состояния заглянуть назад и обновить оценки тех состояний, которые привели к текущему.

Эта модификация известна, как метод TD (λ) обратного обзора (англ. backward view), которая разбивает λ-обновление $G_{t:T}^{\lambda}$ на части, которые применяются на каждом шаге по траектории, составляя своеобразный след из обновлений вдоль траектории.

Этот метод также известен как метод **TD (λ) со следами приемлемости** (англ. eligibility traces).